Agenda

1.	Einleitung	2
2.	Grundlagen	19
3.	Programmiersprachen	31
4.	Automaten	45
5.	Grammatiken	109
6.	Syntaxanalyse	150

Java – Hinweise für die praktischen Hausaufgaben

- Für die praktischen Aufgaben werden wir Ihnen Java-Code zur Verfügung stellen, den Sie um mehrere Methoden erweitern müssen.
- Dieser Code verwendet:
  - Records
  - Generische Typen
  - Type Patterns
  - Streams
- Die folgenden Folien erklären diese Konzepte nochmals kurz.

Java Records

- Java Records sind „Datenklassen“ mit automatisch erzeugtem Konstruktor und Zugriffsmethoden
- Beispiel

```
public record Koordinate(int x, int y) { }
…
Koordinate k = new Koordinate(10,20);          // Konstruktor ist automatisch generiert
System.out.println(k.x()  + ", "  + k.y());    // x() und y() werden automatisch generiert.
```

- Die Properties eines Records können nur durch Aufruf des Konstruktors gesetzt werden. Records sind immutabel!
- toString(), equals(), hashCode() werden automatisch generiert, können aber überschrieben werden.

Generische Typen

- Java unterstützt generische Typen, d.h. Typen, die durch andere Typen („Typparameter“) spezifiziert sind.
- Beispiele:
  Set<String> s = new HashSet<>();
  List<Integer> i = new ArrayList<Integer>(Arrays.*asList*(1,2,3));
- Die Definition eigener generischer Typen ist möglich:

Beispiel:

```
public class MyGenericClass<S, T> {                    // zwei Typparameter
   private S x;
   private T y;

   public MyGenericClass(S x, T y) {this.x = x; this.y = y; }
}
…
MyGenericClass<String, Koordinate>  m = new MyGenericClass<>("MGC", new Koordinate(1,1));
```

Was sind die Vorteile der Nutzung generischer Klassen?

Generische Methoden

- Auch Methoden können Typparameter enthalten
- Beispiel:

```
static  <T> void printStuff(T data){                    // Eingabeparameter vom Typ T, Rückgabetypvoid
    System.out.println("------------");
    System.out.println(data);
    System.out.println("------------");
}
…
printStuff(new Koordinate(5,6));
printStuff(123);
```

- Typparameter können (auch bei Klassen) eingeschränkt werden:
- Beispiel:

```
public class MyClass< T extends Number> {…}          // Typparameter muss Number oder eine Subklasse davon sein.
```

Type Patterns

- Seit Java 21 gibt es ein Feature, um zu prüfen, zu welcher Unterklasse ein Objekt gehört:

```
return switch (item) {
        case Book book -> book.getPrice() * 0.15;
        case CD   cd   -> cd.getPrice() * 0.12;
        case Food food -> food.getPrice() * 0.09;
};
```

- Der Compiler überprüft dabei, dass Sie alle möglichen Untertypen aufgeführt haben!
- Wie löst man das?

- Seit Java 17 kann man für Klassen und Interfaces angeben, welche Klassen diese erweitern bzw. umsetzen:

```
public sealed interface Item permits Book, CD, Food { … }
```

- Achtung: Die erlaubten Klassen müssen dann wiederum **sealed** oder **final** sein.

Streams

- Streams sind Sequenzen von Elementen auf denen Funktionen ausgeführt werden können.
- Streams werden oft aus Collections erzeugt, indem man auf diesen die Methode stream() aufruft.
- Beispiele:

```
List<Integer> i = new ArrayList<>(Arrays.asList(1,2,3));
…
… i.stream().filter( x-> x>2).map(num->2*num)        // Behalte nur die Einträge größer 2 und verdoppele
```

- Häufig verwendete Funktionen auf Streams:
  - filter(), map(), sorted() : Geben wieder einen Stream zurück
  - reduce(), forEach(): Sind terminal, d.h. der Stream ist danach geschlossen.

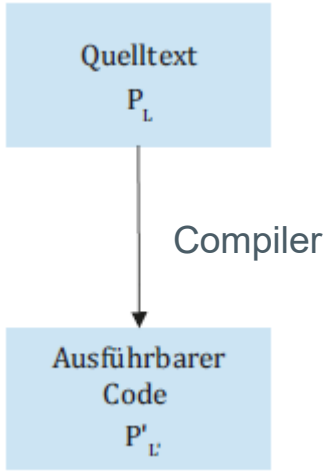
  Achtung: Seiteneffekte in den aufgerufenen Funktionen sollten vermieden werden!

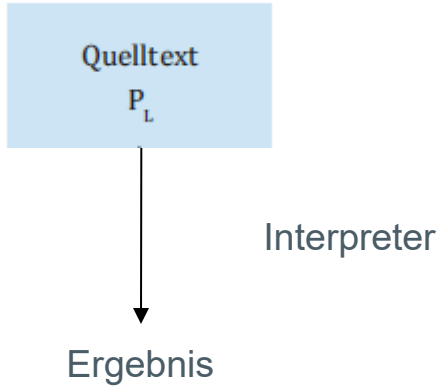
Agenda

1.	Einleitung	2
2.	Grundlagen	19
3.	**Programmiersprachen**	**31**
4.	Automaten	45
5.	Grammatiken	109
6.	Syntaxanalyse	150

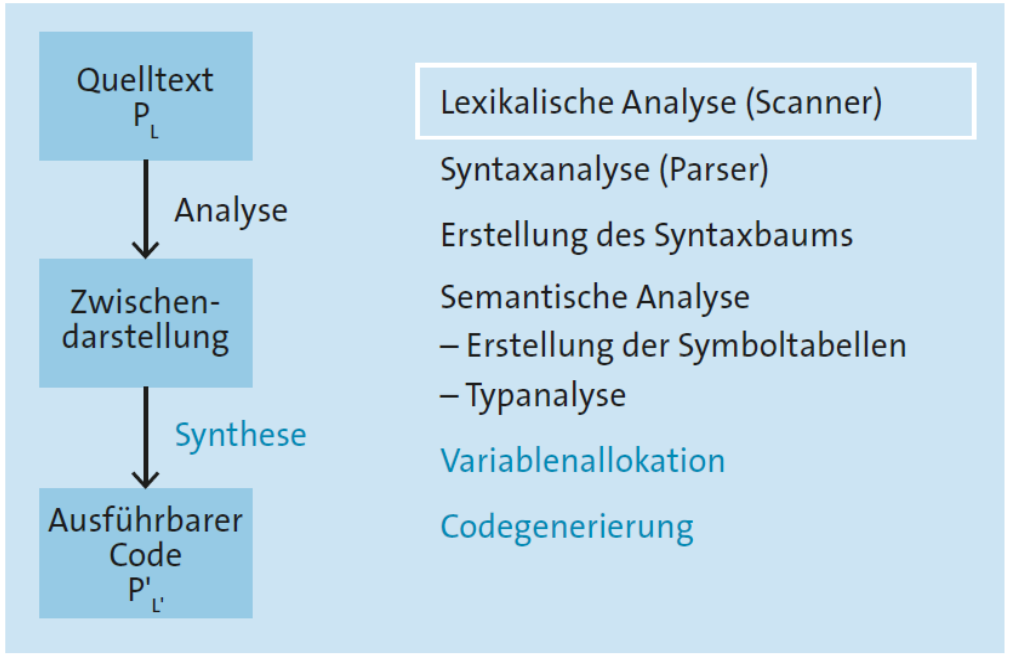
Was ist eine Programmiersprache?

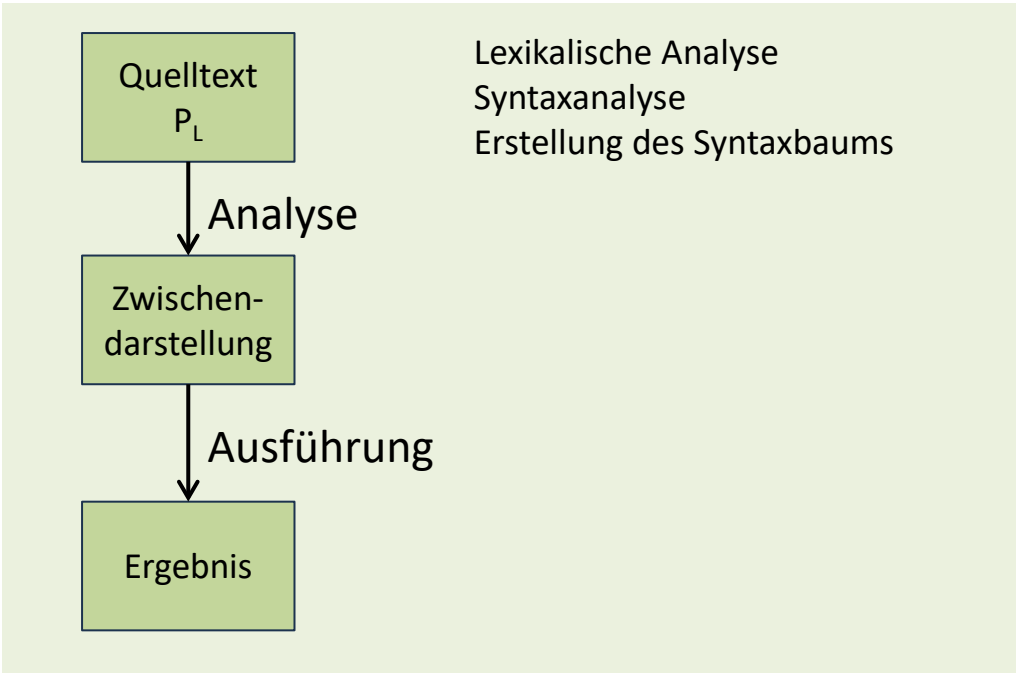
- Eine „**Programmiersprache**“ ist eine Abstraktion des Befehlsatzes von Computern.
- „Programme“, die in einer Programmiersprache L geschrieben sind, können ausgeführt werden, indem Sie
  a) **Kompiliert** werden in eine ausführbare Sprache (z.B. Maschinencode), oder
  b) **Interpretiert** werden.





Phasenmodell für Interpreter und Compiler





Die ersten Phasen sind identisch – oft wird die statische Analyse der Semantik im Interpreter nicht benötigt.

Elemente einer Programmiersprache

- **Lexikalische Elemente**
  Der Zeichenvorrat, aus dem Programme bestehen, wie z.B.:
  - Literale: `42, “hallo”, ‘x’, -3.1415`
  - Bezeichner (Identifier): `x, int, main, args, MeineKlasse`
- **Ausdrücke** (Expressions), die einen Wert repräsentieren:
  `x, 1+x, f(x), x>0`
- **Anweisungen** (Statements)
  - Zuweisungen
  - Konditionale
  - Schleifen
  - Prozeduraufrufe
  - ...

Variablen-Deklaration und -Definition

`int x;`	Deklaration
`x=42;`	Definition
`const float y=3.5;`	Deklaration und Definition

Was macht der Compiler damit?

Immer wenn ein Bezeichner verwendet wird, muss der Compiler auf diese Information zugreifen, um den richtigen Assembler-Code zu erzeugen.
So z.B. benötigt ein `int` weniger Speicherplatz als ein `float`.
Eine Konstante könnte direkt durch ihren Wert ersetzt werden.

Gültigkeitsbereich (‚*Scope*')

Der Gültigkeitsbereich eines Bezeichners ist der Bereich innerhalb des Quelltexts, in dem der deklarierte Bezeichner verwendet werden kann.
Beispiel:

```
fun lokal() {
    var i=1;
    var j=i+1;
    print(j);
}
var i = 0;
{
  var i=42;
  print(i);
}
lokal();
print(j);
```

Lokale Variablen i und j
y=2 !

Globale Variable i

Neue Variable i
Ausgabe: 42

Ausgabe: 2

Error ! Warum?

Gültigkeitsbereich

Scope

= Raum im Quelltext, in dem der Bezeichner verwendet werden kann.

Schlüsselwörter (‘*keywords*’)

Ein Schlüsselwort ist eine fest vorgegebene Zeichenfolge mit einer rein syntaktischen Funktion.

Schlüsselwörter

Beispiele: `if, then, else, while, …`

Keywords

In den meisten Programmiersprachen sind Schlüsselwörter reserviert, d.h. sie dürfen nicht als Bezeichner verwendet werden.

Reserviert oder nicht?

Gegenbeispiel PL/I:

```
IF IF THEN THEN=ELSE
```

Gegenbeispiel COBOL:

```
IF N IS GREATER THAN 0 AND N IS LESS THAN 10 THEN …
```



Ausdrücke (‘*expressions*’)

Ein Ausdruck repräsentiert einen Wert.

Ausdruck

Expression

Repräsentiert einen Wert

Hat einen Typ (meistens)

Beispiele:

```
x
3.1415
x+(2*x-3.1415)
```

Was macht der Compiler damit?

- Überprüfung, ob der Ausdruck “wohl-getypt” ist (Semantische Analyse) – wenn die Sprache statisch getypt ist. TiEL ist dynamisch getypt, sodass `var j=42; j="abc";` möglich ist.
- Generierung von Assembler-Code, der zur Laufzeit den Ausdruck auswertet

Anweisungen (‚Statements‘)

- Anweisung haben einen Effekt, sie ändern entweder den Zustand des Speichers oder machen eine Ausgabe.
- In den meisten Sprachen sind einige Expressions gefolgt von Semikolon Statements.

Java:
- Zuweisungen
- Pre/Post-In/Decrement
- Methodenaufrufe
- Klassenerzeugung (new)

Beispiel: `i=j=2*x;`

TiEL:
- Zuweisungen
- Funktionsaufrufe

Beispiel:
```
fun inc(n) { return n+1; }
inc(3);
```

L-Value und R-Value

Was bedeutet `i=i+1` zur Laufzeit?

Das “rechte“ `i` (R-Value) steht für den aktuellen Wert von `i`, das linke (L-Value) für die Speicherstelle, an der der Wert von `i` gespeichert wird.

Was bedeutet das für einen Compiler?

Bei der Codegenerierung muss man zwischen L-Value und R-Value unterscheiden!

L-Value
=
Adresse

R-Value
=
Wert

Typen

Ein Typ repräsentiert eine Wertemenge zusammen mit den dazugehörigen Operationen.

- Typisiert – Schwach typisiert – Untypisiert
  In den meisten Programmiersprachen gibt es Regeln, die definieren, welche Operationen auf den Werten erlaubt sind.
  Beispiel: `1 + (x > 0)` Erlaubt oder nicht?

- Statische – Dynamische Typisierung
  Bei einer statisch-typisierten Sprache prüft der Compiler die korrekte Verwendung der Typen.
  Bei einer dynamisch-typisierten Sprache erledigt dies das Laufzeitsystem.
  Die meisten Sprachen basieren auf einer Mischform.

Tiny Education Language (TiEL)

TiEL ist eine winzige, dynamische getypte Programmiersprache.

```
fun fib(n) {
  if n == 0 { return 0; }
  elif n == 1 { return 1; }
     else { return fib(n - 1) + fib(n - 2); }
}

var i = 0;
while i < 11 {
  print(i);
  print(fib(i));
  print("");
  i = i + 1;
}
```

TiEL Beispielprogramm (fibonacci.tiel)

TiEL on one page

Statements:

- Zuweisung `x=0;`
- Konditional: `if (x<0) { i=1; } else { i=0; }`
- Schleife: `while (x>1) { x=x-1; }`
- Blockanweisungen: `{ x=1; y=2; }`
- Funktionsaufrufe: `fib(5);`
- Rückgabe: `return false;`
- Funktionen mit call-by-value:

```
fun inc(n){ return n+1; }
```

Ausdrücke:

- Operatoren: `+,-,*,/,<, and, or, not, ==`
- Short-circuit-Semantik
- Der Typ eines Ausdrucks wird inferiert.
- Für Operatoren sind zulässige Operandentypen definiert.
  Beispiel: `x=x+true  // Error`

- Nur eingebaute Datentypen:
  Gleitkommazahlen, Wahrheitswerte, Strings
  Beispiele: `47.11  true  false  "hallo"  nil`
- Variablen müssen deklariert werden und sind dynamisch typisiert.
  Beispiel: `var x = 42;  // Gleitkommazahl`
- Scopes sind statisch und global oder lokal (auch geschachtelt)
- Funktionen sind nur global
- Aufruf des Interpreters

```
java –jar tiel.jar fibonacci.tiel
```

Optionen:

`--ast`	Gibt den abstrakten Syntaxbaum aus
`--tokens`	Gibt die Tokens aus
`--help`	
…	